Evaluación del módulo

Consigna del proyecto 👨🏻‍💻

alkemy

Proyecto: Vue Product Showcase

Evaluación del módulo: DESARROLLO DE APLICACIONES FRONT-END CON FRAMEWORK VUE

Situación inicial 📍

Unidad solicitante: Departamento de E-commerce de una empresa de tecnología ficticia

El equipo de desarrollo del área de e-commerce necesita una aplicación SPA (Single Page Application) que sirva como catálogo interactivo de productos. Se busca un sistema moderno, dinámico y responsive que permita a los usuarios visualizar información de productos, filtrar por categorías y ver detalles individuales. Actualmente, el equipo utiliza una API interna para exponer los productos, pero no cuentan con una interfaz robusta que consuma esos datos ni gestione el estado de forma centralizada. Además, buscan asegurar la calidad de la solución con pruebas automatizadas y dejar abierta la posibilidad de escalar la aplicación a móvil o escritorio.

Nuestro objetivo 📋

Desarrollar una aplicación Vue moderna, optimizada y mantenible que permita:

- Visualizar un catálogo de productos obtenidos desde una API REST.
- Utilizar componentes reutilizables y bien organizados.
- Gestionar el estado global con Vuex.
- Incorporar pruebas unitarias y e2e.
- Integrar una librería UI y considerar Nuxt o Quasar según el enfoque del proyecto.

La aplicación deberá ser clara, accesible, modular y escalable para futuras mejoras.

Requerimientos 🤝

En este apartado se deben listar las funcionalidades y especificaciones técnicas necesarias para cumplir con los objetivos planteados. Es fundamental detallar tanto los **aspectos generales** como los **específicos**, proporcionando al

participante una guía clara de las expectativas técnicas y funcionales del proyecto.

Paso a paso 👣

Este proyecto refiere exclusivamente al **módulo 7**: *DESARROLLO DE APLICACIONES FRONT-END CON FRAMEWORK VUE*, y se compone de **5 etapas (lecciones)**, las cuales podrás avanzar de forma progresiva y escalonada con la ayuda de los manuales teóricos y los contenidos desarrollados en las clases en vivo.

Ten en cuenta de invertir **tiempo asincrónicos** para el desarrollo de cada etapa a modo de poder finalizar el módulo y realizar la entrega formal de tu propuesta. Cualquier consulta que surja compártela en los espacios sincrónicos para resolver las dudas en equipo.

A continuación encontrarás las consignas y tareas a desarrollar:

Lección 1: *Componentes y ciclo de vida*

🎯 **Objetivo:** Iniciar la base del proyecto con estructura de componentes reutilizables.

📋 **Tareas a desarrollar**:

- Configurar el proyecto con Vue CLI.
- Crear el componente `<ProductCard>` para mostrar un producto.
- Implementar al menos un ciclo de vida (`mounted`, `created`, etc.).
- Diseñar la estructura base de la app: `<App>`, `<Header>`, `<Footer>`, `<ProductList>`.

➜ Esta lección establece las bases visuales y de estructura para el resto del proyecto.

Lección 2: Consumo de datos desde una API

🎯 **Objetivo**: Incorporar productos dinámicos mediante consumo de API.

📋 **Tareas a desarrollar**:

- Integrar Axios y obtener datos desde una API (pública o mock con JSON-server).
- Mostrar productos dinámicamente en <ProductList>.

    - Incluir gestión de errores y carga (loading, error, empty).
    - Agregar filtro simple por categoría.

➜ Esta lección conecta los componentes del paso anterior con datos reales y lógica.

Lección 3: Almacenamiento de estado en Vuex

🎯 **Objetivo:** Centralizar el estado y mejorar la arquitectura de datos.

📋 **Tareas a desarrollar**:

- Configurar Vuex y separar en módulos (productos, filtros, favoritos).
- Mover el consumo de API a acciones Vuex.
- Usar getters para computar productos filtrados.
- Conectar los componentes visuales con el estado centralizado.

➜ Esta entrega optimiza el flujo de datos y prepara la app para escalar.

Lección 4: Pruebas en Vue

🎯 **Objetivo:** Asegurar calidad de componentes y flujos clave.

📋 **Tareas a desarrollar**:

- Escribir 2 pruebas unitarias con Vue Test Utils + Jest:
    - Render correcto de <ProductCard>
    - Respuesta visual ante error de API
- Crear 1 prueba end-to-end con Cypress o Nightwatch:
    - Usuario filtra productos y ve resultados.

➜ Esta etapa valida el comportamiento de lo construido hasta aquí.

Lección 5: Librerías y Frameworks Complementarios

🎯 **Objetivo**: Aplicar diseño visual profesional y mejorar arquitectura.

📋 **Tareas a desarrollar**:

- Elegir una librería UI (Vuetify, Element Plus, etc.) y aplicarla.
- Estilizar <ProductCard>, botones, inputs, etc.
- (Opcional) Migrar a Nuxt o Quasar y justificar por qué.
- Aplicar diseño responsive y tema claro/oscuro.

➜ Esta lección refina la experiencia visual del usuario final.

¿Qué vamos a validar? 🔍

- Arquitectura escalable por componentes.
- Integración y visualización correcta de datos.
- Gestión efectiva del estado global.
- Cobertura básica de pruebas (unitarias y e2e).
- Uso profesional de librerías UI.
- Entrega funcional, limpia y documentada.

Referencias 🦺

Vue.js Docs	Axios Docs	Vue Test Utils	Vuetify
Vuex Docs	Jest	Cypress	Quasar Nuxt

Entregables ✅

- Código fuente en GitHub con estructura clara.
- `README.md` con instrucciones de instalación y justificaciones técnicas.
- Evidencias de pruebas ejecutadas.
- Demo funcional o capturas del producto final.

Portafolio 💼

Este proyecto es ideal para mostrar en tu portafolio como ejemplo de:

- Aplicación SPA con Vue.
- Integración de arquitectura moderna (Vuex, componentes, pruebas).
- Criterios de calidad, diseño y documentación profesional.

Recomendamos publicar capturas, enlazar al repo y redactar un breve resumen explicando el enfoque y decisiones técnicas.

¡Éxitos!

Nos vemos más adelante

alkemy